

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering
Software Engineering - Level 3 Mini Project (UE24CS341A)

Software Requirements Specification

for

QuizSphere - Online Quiz Assessment and Analytics System

Version 1.0.1

Prepared by Team 12: Arundhathi K, Aryan Revankar, Ashwin Venkatesh, Chaturya Prabhakar Reddy

PES University, Department of Computer Science and Engineering

Date created: 06-09-2026

Revision History

Name	Date	Reason for Changes	Version
Team 12	07-09-2026	Initial version	1.0
Team 12	01-10-2026	Expanded FR-020 to explicitly cover the post-attempt student-facing weak-topic view; renamed UC-06 to “View Result and Leaderboard and Weak-Topic Summary”; updated Section 2.2 Product Functions to match. Added FR-022 (administrator account enable/disable) so the ACCOUNT_DISABLED error used in the SAD and Test Plan traces to a requirement. Corrected Section 12 Tool Stack text to say one row is marked not applicable, matching the table.	1.0.1

1. Introduction

1.1 Purpose

This document specifies the functional, non-functional and security requirements for QuizSphere, the Online Quiz Assessment and Analytics System proposed in the Deliverable 1 synopsis. It expands the objectives and functional feature list already submitted into requirements that can be built and tested against.

1.2 Intended Audience and Reading Suggestions

This document is written for the four members of Team 12, for the course instructor and evaluators reviewing the project, and for anyone who later maintains the codebase. Sections 1 and 2 give context, Sections 3 to 9 are the actual requirements, and Sections 10 to 13 cover validation planning, the project plan, the tool stack and reference appendices.

1.3 Product Scope

QuizSphere is a locally run C and C++ application for a department or training cell. An instructor builds a tagged question bank, assembles a timed quiz from it with a chosen scoring scheme, and students

attempt it with instant auto-grading. Instructors and admins then see topic level and student level analytics from the results. A detailed in-scope and out-of-scope breakdown is given in Section 3.

1.4 References

PES University Software Engineering Project Guidelines, UE24CS341A. QuizSphere Deliverable 1 Synopsis, Team 12, Group 12.

2. Overall Description

2.1 Product Perspective

QuizSphere is a new, self-contained product. It is not a follow-on to an existing system and does not integrate with any larger university system. It is a single local application built and used independently by the department or training cell that adopts it.

2.2 Product Functions

- Manage a tagged question bank (add, edit, delete, bulk import/export).
- Create and configure quizzes with a time limit and a scoring scheme.
- Let students attempt quizzes in practice or graded mode, with auto-grading.
- Rank graded attempts on a leaderboard.
- Produce topic level and student level analytics, shown to instructors and admins as reports and to students directly after a graded attempt, and export result reports.
- Let an administrator manage user accounts and roles, including enabling or disabling an account.

2.3 User Classes and Characteristics

User Class	Privileges	Characteristics
Student	Register/login, attempt assigned quizzes, view own result and leaderboard position	Non-technical, uses the system mainly around test windows
Instructor	Create and tag questions, assemble and configure quizzes, view class analytics	Moderately technical, frequent use while preparing content
Administrator	Manage users and roles, oversee question bank import/export, view system reports	Higher privilege, infrequent but important actions

2.4 Operating Environment

QuizSphere runs as a native console application on a single machine, Windows or Linux, with no dependency on internet connectivity. Persistence is local, through SQLite3 or structured files on the same machine. No server, browser or mobile platform is targeted.

2.5 Design and Implementation Constraints

- Must be implemented in C and C++, the technology assigned to the team.
- Must use the SQLite3 C API or plain file I/O only, no external database server.

- Must be delivered incrementally across a single semester.
- Must stay within the Easy complexity budget agreed at the synopsis stage, so no networked or concurrent multi-user features.

2.6 Assumptions and Dependencies

- Assumes each user runs the application on their own machine with the SQLite3 library available at compile time.
- Assumes question content is supplied by instructors and is factually correct, since QuizSphere validates format, not subject-matter accuracy.
- Depends on GoogleTest and cppcheck being available in the CI environment used for testing and static analysis.

3. Scope

In Scope	Out of Scope
Registration and role based login with hashed password storage	Subjective or essay question grading
Question bank: create, edit, delete, tag by subject/difficulty/type	Webcam or lockdown-browser style proctoring
Quiz creation with time limit, question count, randomised pooling	Payment, certification or accreditation
Configurable scoring per quiz, including negative marking	Native mobile app
Timed attempt engine with shuffled questions and options	Integration with an external LMS
Practice mode and graded mode (single attempt, logged)	Real-time multi-user or networked quiz sessions
Auto-grading with instant, itemised result and review	Cloud deployment or multi-campus scaling
Leaderboard and topic/student performance analytics	AI-generated question authoring
Item analysis: per-question difficulty and per-topic breakdown	
CSV export of per-student result reports	
Local persistence (SQLite or structured file storage)	
Administrator management of user accounts, roles, and account enable/disable	

4. External Interface Requirements

4.1 User Interfaces

The system is operated through role based console menus, one flow each for Student, Instructor and Administrator. Every screen shows the logged in user's name and role, and every destructive action, such as deleting a question or removing a user, asks for a yes or no confirmation before proceeding.

4.2 Software Interfaces

The application talks to a local SQLite database file through the SQLite3 C API. It also reads and writes plain CSV files for question bank import/export and for result report export.

4.3 Communications Interfaces

None. The system runs as a single local program and does not require a network connection, which keeps it within the Easy complexity budget agreed in the synopsis.

5. Analysis Models

At this stage the analysis model is expressed as the use case list below. A use case diagram and an entity relationship diagram will be added in the Design deliverable, built directly from these use cases and from the field layout in Appendix B.

Use Case ID	Use Case	Primary Actor	Precondition	Outcome
UC-01	Register / Login	Student, Instructor, Admin	Application available	Authenticated session with correct role
UC-02	Manage Question Bank	Instructor	Logged in as instructor	Question added, edited or removed
UC-03	Import / Export Questions	Instructor	Logged in as instructor	Question bank updated, or CSV file written
UC-04	Create Quiz	Instructor	Logged in, question bank has items	Quiz created with time limit and scoring rules
UC-05	Attempt Quiz	Student	Logged in, quiz assigned	Attempt submitted and graded, or saved as practice
UC-06	View Result and Leaderboard and Weak-Topic Summary	Student	Attempt submitted	Result and rank and per-topic weak areas displayed
UC-07	View Analytics	Instructor, Admin	Logged in, at least one graded attempt exists	Item analysis and topic report displayed
UC-08	Export Result Report	Instructor, Admin	Logged in, graded attempts exist	CSV transcript file written
UC-09	Manage Users	Administrator	Logged in as admin	User account, role, or enabled/disabled status updated

6. Functional Requirements

ID	Requirement	Priority	Verification
FR-001	The system shall allow a new user to register with a unique username and a role of Student or Instructor.	High	Functional test
FR-002	The system shall authenticate a user against the stored password hash before starting a session.	High	Functional test
FR-003	The system shall store every password as a SHA-256 hash and never in plaintext.	High	Code review
FR-004	The system shall let an instructor add, edit or delete a question, with a subject tag, a difficulty tag and a question type.	High	Functional test
FR-005	The system shall reject a question that is missing a subject tag, a correct answer, or has fewer than two options.	High	Negative test
FR-006	The system shall import questions in bulk from a CSV file, skipping and reporting any row that fails validation.	Medium	Functional test
FR-007	The system shall export the current question bank, or a filtered subset of it, to a CSV file.	Medium	Functional test
FR-008	The system shall let an instructor create a quiz by choosing a time limit, a question count and a subject or difficulty filter to pool questions from.	High	Functional test
FR-009	The system shall let an instructor set a scoring scheme per quiz, including marks per question and an optional negative mark per wrong answer.	High	Functional test
FR-010	The system shall shuffle the order of questions and the order of options within each question, independently for every attempt.	High	Functional test
FR-011	The system shall enforce the quiz time limit and auto-submit the attempt when time runs out.	High	System test
FR-012	The system shall let a student navigate between questions and flag a question for review before submitting.	Medium	Functional test
FR-013	The system shall let a student choose practice mode or graded mode before starting an attempt.	High	Functional test
FR-014	In practice mode, the system shall show the correct answer immediately after each question and shall not record the attempt on the leaderboard.	Medium	Functional test
FR-015	In graded mode, the system shall allow only one attempt per student per quiz and shall log that attempt permanently.	High	Negative test
FR-016	The system shall auto-grade a submitted attempt using the quiz's configured scoring rules, including negative marking where set.	High	Unit and system test
FR-017	The system shall show the student an itemised result: each question, the student's answer, the correct answer and marks awarded.	High	Functional test
FR-018	The system shall rank graded attempts on a leaderboard per quiz, per topic and all time.	Medium	Functional test
FR-019	The system shall compute a per-question difficulty index as the percentage of attempts that answered it correctly.	Medium	Unit test
FR-020	The system shall compute a per-topic performance breakdown, showing average score by subject tag, for a student and for a class. For the student's own attempt, this breakdown shall also be displayed to the student immediately after a graded	Medium	Unit test

	attempt, alongside the itemised result (FR-017), highlighting the topics where their score was weakest.		
FR-021	The system shall export a per-student result transcript to CSV on request.	Low	Functional test
FR-022	The system shall let an Administrator enable or disable a user account; the system shall refuse login to a disabled account with the ACCOUNT_DISABLED error.	Medium	Functional / Negative test

7. Non-Functional Requirements

ID	Category	Measurable Requirement	Verification
NFR-001	Performance	A quiz of 20 questions shall load and start within 2 seconds on the target machine.	Timed test run
NFR-002	Usability	A new student shall be able to log in and start an assigned quiz within 3 menu steps.	Usability walkthrough
NFR-003	Reliability	An attempt that is auto-submitted on timeout shall be graded the same way as a manually submitted one.	Negative and integration test
NFR-004	Maintainability	Each functional area (auth, question bank, quiz engine, grading, analytics) shall be a separate module with its own header and source files.	Code review
NFR-005	Testability	Every module's core logic shall be callable and testable without going through the console menu.	Unit test
NFR-006	Data Integrity	A graded attempt's stored answers and score shall not be editable once submitted.	Integration test

8. Security Requirements

ID	Security Requirement	Validation Approach
SEC-001	Passwords shall never be stored or logged in plaintext.	Code and configuration review
SEC-002	Only Instructor or Admin roles shall be able to create, edit or delete questions and quizzes.	Authorization test
SEC-003	Only Admin shall be able to create, edit or remove user accounts and roles.	Authorization test
SEC-004	The system shall validate and sanitise every user supplied input field before storing or processing it.	Negative and security test
SEC-005	A CSV import shall be validated row by row and shall never overwrite the question bank if the file is malformed.	Negative test
SEC-006	Failed login attempts shall be logged with a timestamp, without logging the attempted password.	System test

9. Business Rules

- Only a question marked available can be added to a new quiz.
- A quiz cannot be created with zero questions or a zero minute time limit.
- A graded attempt cannot be reattempted once it has been submitted.
- Practice attempts are never shown on a leaderboard and never count toward analytics.
- A negative marking scheme cannot bring a single question's score below zero.
- Only an Instructor or Admin can disable a question so it stops appearing in new quizzes.
- Only an Administrator can disable or enable a user account; a disabled account is refused login (FR-022).

10. Validation Specification: Initial Planning

At this stage the validation plan is about what will eventually be tested, not the actual results. Full execution and pass/fail evidence belongs to the Implementation and Testing deliverable.

Requirement Category	Planned Validation	Example
Functional	Black box functional tests	Create a quiz and attempt it end to end
Security	Authorization and negative tests	Student attempts to open the question bank editor
Performance	Timed test runs	Load and start a 20 question quiz
Usability	Task based walkthrough	New student takes their first quiz
Reliability	Timeout and retry scenarios	Auto-submit on timer expiry
Unit	Automated unit tests with coverage	Scoring calculation, difficulty index calculation

11. Project Plan

This plan carries over the sprint roadmap from the synopsis, broken into work packages with estimated effort.

Work Package	Tasks	Owner	Effort (person-days)	Output
WP1	Requirements and use cases	All	3	SRS
WP2	Auth module: registration, login, roles, hashing	Arundhathi	6	Working feature
WP3	Question bank module and CSV import/export	Aryan	6	Working feature
WP4	Quiz configuration and timed attempt engine	Ashwin	5	Working feature
WP5	Auto-grading and practice/graded mode (Ashwin); leaderboard (Chaturya)	Ashwin / Chaturya	5	Working feature

WP6	Item analysis, analytics and CSV result export	Chaturya	5	Working feature
WP7	Testing, CI and security validation	All	5	Test and validation evidence
WP8	Final validation and report	All	4	Final report

12. Tool Stack

This maps the project onto the tool infrastructure the course guidelines list (Git, GitHub, CI/CD, Docker, testing tools, Jira or similar, SonarQube or similar). One row is marked not applicable, with the reason given, rather than forced in to look complete.

Activity	Tool	Evidence to Submit
Version control	Git + GitHub	Repository URL, commit history
Agile planning / bug tracking	GitHub Projects and Issues (Jira-equivalent)	Backlog, user stories, sprint board, issues linked to tasks
CI/CD	GitHub Actions (Jenkins-equivalent): build and test on every push/PR	Workflow run logs, pipeline screenshots
Containerization	Docker, to pin the build environment (compiler, SQLite3, gtest, cppcheck versions) so it builds the same way on every member's machine	Dockerfile, build/run evidence
Testing	GoogleTest (gtest)	Test results, coverage (gcov/lcov)
Static analysis	cppcheck (SonarQube-equivalent)	Analysis report
Design / UML	Draw.io or PlantUML	Architecture and UML diagrams
API testing	Not applicable: QuizSphere has no network API, see Section 4.3	Not applicable

13. Appendices

Appendix A: Glossary

Term	Meaning
Quiz	A named, timed set of questions with a scoring scheme, created by an instructor
Attempt	One student's run through a quiz, in practice or graded mode
Item	A single question in the question bank
Difficulty index	Percentage of attempts that answered a given item correctly

RBAC	Role based access control, restricting actions by user role
------	---

Appendix B: Field Layout

Sample fields for the questions table.

Field	Length	Data Type	Description	Mandatory
question_id	36	String	Unique question identifier	Y
subject_tag	40	String	Subject or topic label	Y
difficulty	10	Enum	Easy, Medium or Hard	Y
question_text	500	String	The question itself	Y
correct_option	5	String	Index or label of the correct option	Y
is_available	1	Boolean	Whether the question can be added to new quizzes	Y

Appendix C: Requirement Traceability Matrix (sample)

Architecture and design references will be filled in once the Design deliverable is complete.

Req ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID
FR-002	Login authentication	TBD	TBD	auth_module.*	UT-AUTH-01
FR-010	Shuffle questions and options	TBD	TBD	quiz_engine.*	UT-QE-01
FR-016	Auto-grading with scoring rules	TBD	TBD	grading_module.*	UT-GRADE-01
FR-019	Difficulty index calculation	TBD	TBD	analytics_module.*	UT-ANLY-01
FR-020	Per-topic breakdown, incl. post-attempt student view	TBD	TBD	analytics_module.*	UT-ANLY-02
FR-022	Administrator account enable/disable	TBD	TBD	auth_module.*	UT-AUTH-06

Software Architecture and Design Specification

Project: QuizSphere - Online Quiz Assessment and Analytics System

Version: 1.1

Authors: Team 12: Arundhathi K, Aryan Revankar, Ashwin Venkatesh, Chaturya Prabhakar Reddy

Date: 02-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	01-10-2026	Team 12	Initial architecture and design, built from the Deliverable 1 synopsis and Deliverable 2 SRS
1.1	01-10-2026	Team 12	Listed all ten core features in Section 1.2 Scope (previously only eight were named); disambiguated “ten committed functional features” in Section 3.7 from the SRS's 21 functional requirements; corrected the component diagram (Figure 1) so the Question Bank and Analytics modules reach SQLite/CSV only through the Data Access Layer, matching Section 3.4's text; added FR-022 to the traceability table so ACCOUNT_DISABLED (Section 4.3) traces to an SRS requirement; fixed the missing hyphen in SEC-006 and the run-together “1.3 Audience” heading in Section 1.

Approvals

Role	Name	Signature / Date
Course Instructor	Dr. Sapna V M	

1. Introduction

1.1 Purpose

This document specifies the architecture and design of QuizSphere, the Online Quiz Assessment and Analytics System. It builds directly on the requirements in the Deliverable 2 SRS, showing how those requirements become an implementable module structure, and gives the design detail Deliverable 4 (Implementation and Testing) will be built against.

1.2 Scope

Covers the architecture and design of all ten core features from the Deliverable 1 synopsis and SRS:

- 1. Registration and role based login (Student/Instructor/Administrator), with SHA-256 password hashing
- 2. Question bank management: add, edit, delete and tag questions by subject, difficulty and type
- 3. Bulk question import and export via CSV

- 4. Quiz creation and configuration: time limit, question count, randomised pooling, and a configurable scoring scheme
- 5. Timed quiz attempt engine: per-attempt shuffling of questions and options, navigation, and auto-submit on timeout
- 6. Practice mode and graded mode
- 7. Auto-grading with an instant, itemised result and answer key review
- 8. Leaderboard and ranking, per quiz, per topic and all-time
- 9. Item analysis and weak-topic analytics (difficulty index, per-topic breakdown)
- 10. Exportable per-student result report (CSV)

It does not cover subjective grading, proctoring, or networked multi-user play, which are out of scope as stated in SRS Section 3.

1.3 Audience

The four members of Team 12, the course instructor and evaluators, and anyone who maintains the codebase after this semester.

1.4 Definitions

Quiz, Attempt, Item, Difficulty index, RBAC: defined in SRS Appendix A. SAD, UML, STRIDE, CRUD: standard software engineering terms used in this document.

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture: the layers, the modules, the pattern chosen and why, the technology stack, and the main risks and security considerations. Section 4 goes one level deeper into design: sequence diagrams for the two most important flows, the internal module interfaces, error handling, and the console UX. Section 5 has supporting reference material.

2.2 Related Documents

QuizSphere Deliverable 1 Synopsis, Team 12, Group 12. QuizSphere Deliverable 2 Software Requirements Specification v1.0.1, Team 12, Group 12.

3. Architecture

3.1 Goals and Constraints

Goals: correct auto-grading, a quiz that starts within 2 seconds (NFR-001), and a codebase that stays maintainable across four people working in parallel (NFR-004). Constraints, carried over from SRS Section 2.5: must be built in C and C++, must use the SQLite3 C API or plain file I/O with no external database server, must be delivered incrementally across one semester, and must stay within the Easy complexity budget, which rules out networking or concurrent multi-user access.

3.2 Stakeholders and Concerns

Stakeholder	Concern
Students	A quiz that starts fast, grades fairly, and shows them what to study next
Instructors	Reusable question bank, reliable auto-grading, class-level analytics
Course evaluators	Clean module boundaries, traceability from requirement to code to test
Team 12	A codebase four people can work on in parallel without constant merge conflicts

3.3 Component (UML) Diagram

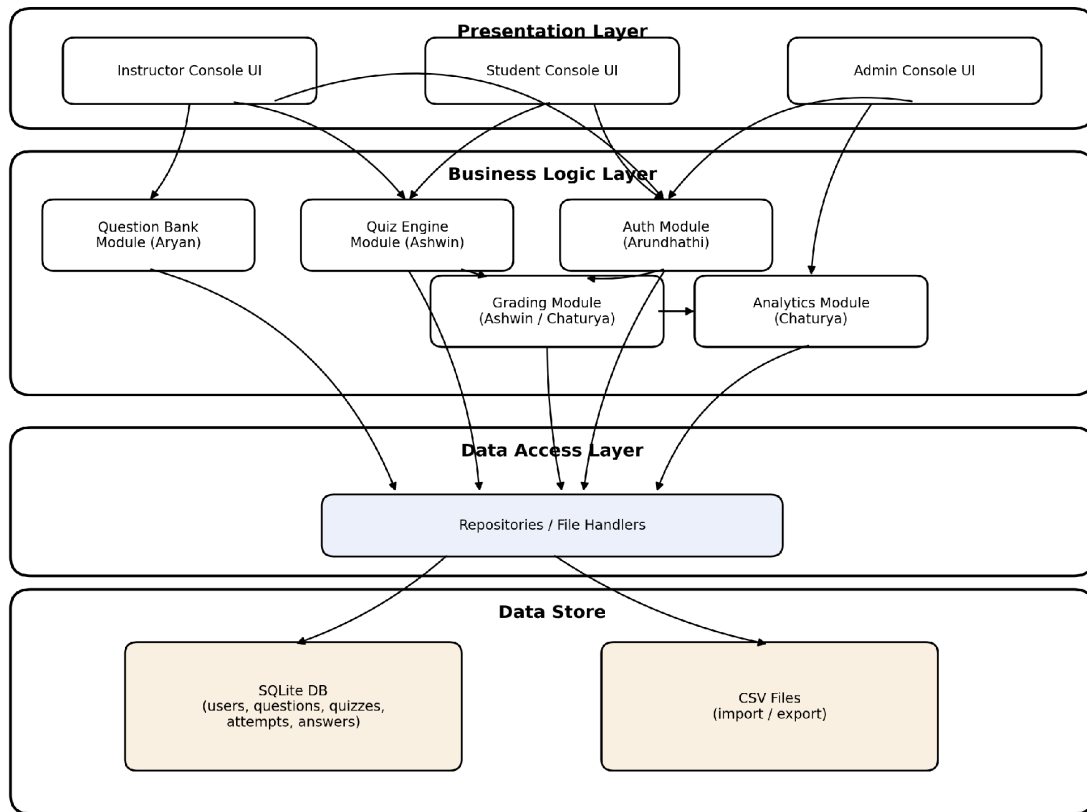


Figure 1: QuizSphere component diagram, showing the presentation, business logic, data access and data store layers. Every business-logic module reaches SQLite or the CSV files only through the Data Access Layer (corrected in v1.1 – the Question Bank and Analytics modules no longer draw directly to CSV Files).

3.4 Component Descriptions

- Console UI (Student, Instructor, Admin): role based menus, input validation at the UI boundary, confirmation prompts before destructive actions.
- Auth Module: registration, login, SHA-256 password hashing, role based session checks used by every other module.
- Question Bank Module: question CRUD, subject/difficulty/type tagging, CSV import and export.

- Quiz Engine Module: quiz creation and configuration, question pooling, per-attempt shuffling of questions and options, the timer, and practice/graded mode handling.
- Grading Module: applies a quiz's scoring rules, including negative marking, to a submitted attempt and produces the itemised result.
- Analytics Module: leaderboard ranking, per-question difficulty index, per-topic performance breakdown, the post-attempt weak-topic summary shown to the student, and CSV result export.
- Data Access Layer: repositories and file handlers that are the only code allowed to talk to SQLite or the filesystem directly.
- Data Store: the SQLite database file, plus CSV files used only for import and export.

3.5 Chosen Architecture Pattern and Rationale

A layered architecture was chosen: Presentation, Business Logic, Data Access, Data. This gives each of the four owned modules (Auth, Question Bank, Quiz Engine, Grading plus Analytics) a clear home in the Business Logic layer, lets every module be unit tested without going through the console menu (NFR-005), and keeps the data access code in one place so swapping the flat-file fallback for SQLite, or back, touches only one layer.

A client-server or microservice split was considered and rejected. SRS Section 2.5 already rules out networked or concurrent multi-user features for this semester, so splitting the system into services would add deployment and communication complexity with no corresponding requirement to justify it.

3.6 Technology Stack and Data Stores

C and C++ with CMake, SQLite3 via its C API for persistence (with a flat-file fallback for development), GoogleTest for unit tests, cppcheck for static analysis, and GitHub Actions for CI. This is unchanged from the Tool Stack in SRS Section 12.

3.7 Risks and Mitigations

Risk	Mitigation
SQLite file left in a half-written state if the program exits mid-write	Wrap every multi-step write (for example, storing an attempt plus its answers) in a single transaction
A malformed CSV import corrupts the existing question bank	Validate every row before writing anything; reject the whole import on a structural failure (SEC-005)
Four people editing the same SQLite file during development causes confusing local state	Each member works against their own local database file during development; a single shared file is only used for integration testing and the final demo
Scope creep past the Easy complexity budget agreed at the synopsis stage	Any feature beyond the ten committed functional features (Deliverable 1 Synopsis Section 6 — distinct from the SRS's 21 individually numbered functional requirements) is logged under Open Issues (Section 4.6) rather than built ad hoc

3.8 Traceability to Requirements

This fills in the Architecture Reference and Design Reference columns that were left as TBD in SRS Appendix C.

Req ID	Brief Description	Architecture Reference	Design Reference
FR-002	Login authentication	Auth Module, Data Access Layer	Section 4.3, Login Interface
FR-010	Shuffle questions and options	Quiz Engine Module	Section 4.2, Attempt Quiz sequence
FR-016	Auto-grading with scoring rules	Grading Module	Section 4.3, GradeAttempt Interface
FR-019	Difficulty index calculation	Analytics Module, Data Access Layer	Section 3.4, Analytics Module
FR-020	Per-topic breakdown, including post-attempt student view	Analytics Module	Section 4.2, Attempt Quiz sequence, step 9-10
FR-022	Administrator account enable/disable	Auth Module, Data Access Layer	Section 4.3, Login Interface (ACCOUNT_DISABLED)

3.9 Security Architecture

QuizSphere has no network surface (SRS Section 4.3), so the STRIDE categories that apply are narrower than for a networked system, but still worth stating explicitly.

STRIDE Category	Applies How	Mitigation
Spoofing	Someone attempts to log in as another user	Password required and checked against its stored hash (FR-002)
Tampering	A graded attempt is edited after submission	Graded attempts are locked once submitted, enforced at the Grading Module, not just the UI (NFR-006)
Repudiation	A student denies having taken a graded attempt	Every graded attempt is logged permanently with a timestamp (FR-015)
Information Disclosure	Passwords exposed through storage or logs	SHA-256 hashing, never logging a plaintext password, even on a failed login (SEC-001, SEC-006)
Denial of Service	A malformed CSV import crashes the question bank	Row-by-row validation before any write (SEC-005)
Elevation of Privilege	A student reaches Instructor or Admin-only actions	Role check at the start of every Instructor or Admin function, not only hidden in the UI (SEC-002, SEC-003)

4. Design

4.1 Design Overview

Each module in Section 3.4 is designed as a set of plain functions operating on a small number of structs (User, Question, Quiz, Attempt), with no module calling another module's internals directly, only its declared interface. This keeps the design consistent with NFR-004 (each functional area is a separate module with its own header and source files) and NFR-005 (every module is testable without the console menu).

4.2 UML Sequence Diagrams

Two flows specific to this project: attempting a graded quiz, and creating a quiz.

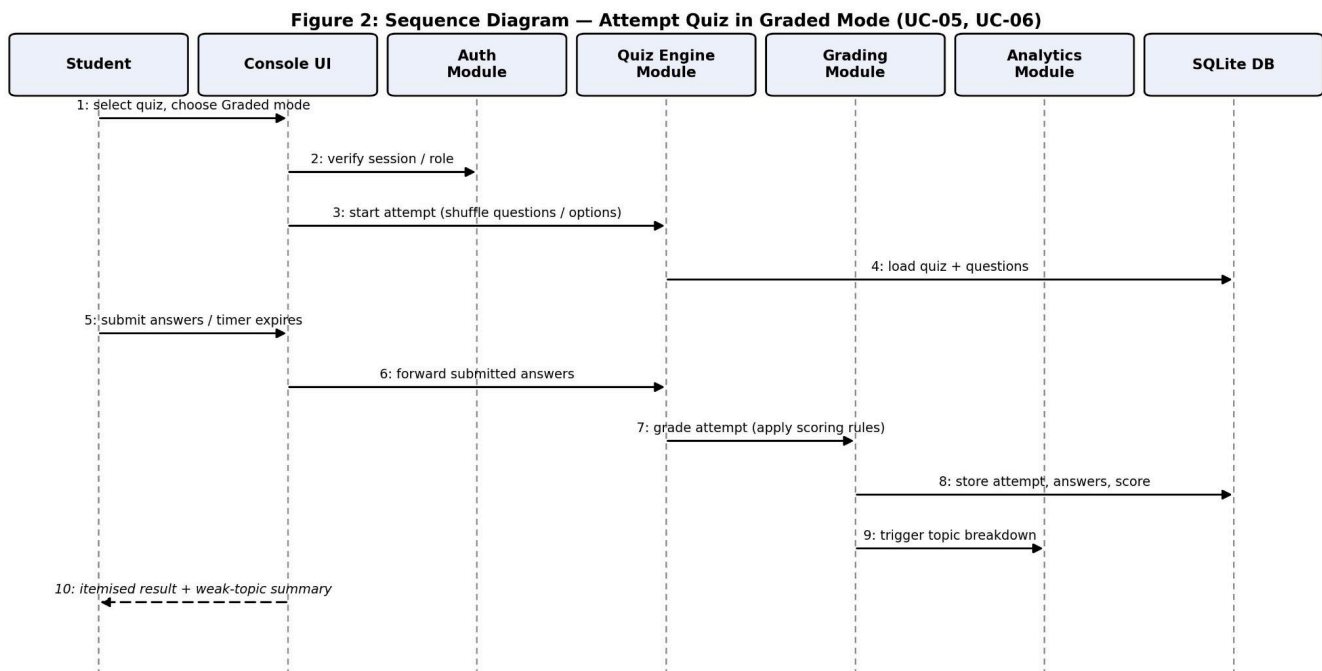


Figure 2: Sequence diagram for Attempt Quiz in graded mode (UC-05, UC-06). Student selects quiz, chooses graded mode; Console UI verifies session/role with Auth Module; Quiz Engine starts the attempt (shuffling questions/options); on submit, Quiz Engine forwards answers to the Grading Module, which grades the attempt and stores it via the Data Access Layer; the Grading Module triggers the Analytics Module for the topic breakdown; the student receives the itemised result and weak-topic summary.

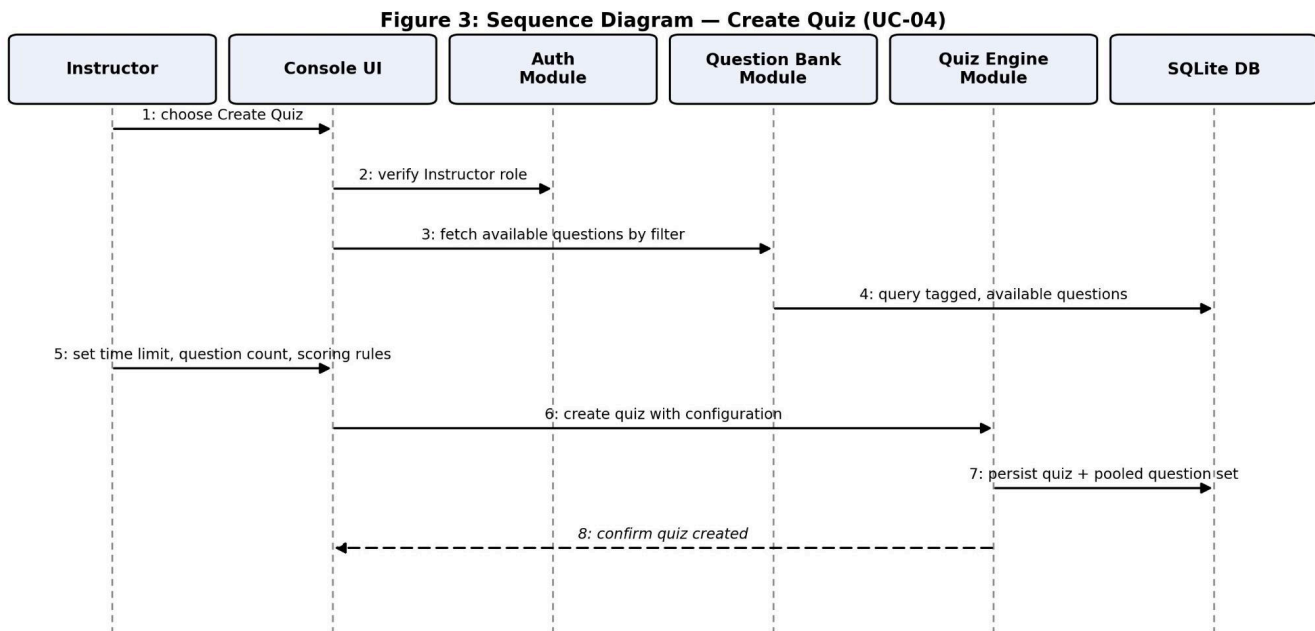


Figure 3: Sequence diagram for Create Quiz (UC-04). Instructor chooses Create Quiz; Console UI verifies the Instructor role with Auth Module; the Question Bank Module is queried for tagged, available questions; the Instructor sets the time limit, question count and scoring rules; the Quiz Engine persists the quiz and its pooled question set via the Data Access Layer.

4.3 Module Interface Design

QuizSphere has no network API (SRS Section 4.3), so this section defines internal module interfaces, function level, instead of REST endpoints. Two representative interfaces are detailed below.

Interface: Auth Module

Function: Login(username, password)

Request: username (string), password (string, plaintext, hashed internally before comparison)

Response: Session{user_id, role, logged_in_at} on success

Errors: INVALID_CREDENTIALS (unknown username or wrong password), ACCOUNT_DISABLED (account disabled by an Administrator, FR-022)

Interface: Grading Module

Function: GradeAttempt(quiz_id, student_id, answers)

Request: quiz_id (string), student_id (string), answers (list of question_id to selected_option)

Response: Result{score, itemised_breakdown, topic_breakdown} on success

Errors: ATTEMPT_ALREADY_SUBMITTED (FR-015), QUIZ_NOT_FOUND, TIME_EXPIRED (still grades, per FR-011, but flags the attempt as auto-submitted)

4.4 Error Handling, Logging and Monitoring

Every module returns a result type that is either a value or a named error code, rather than throwing exceptions across module boundaries, so the console UI can show one consistent error message format. No password, in plaintext or hashed form, is ever written to a log file (SEC-006). Failed logins and rejected CSV rows are logged with a timestamp and reason. Since there is no running service to monitor, monitoring here means the CI pipeline: GitHub Actions reports build status and test pass rate on every push, which stands in for uptime monitoring on a local, single-user application.

4.5 UX Design

The interface is a role based console menu (SRS Section 4.1): numbered options, the logged in user's name and role shown on every screen, a yes or no confirmation before any destructive action, and a visible countdown during a timed attempt. Error messages name the problem and the fix in one line, for example "Question rejected: needs at least two options," rather than a raw error code.

4.6 Open Issues and Next Steps

- Bloom's-taxonomy-based question tagging was considered, after a discussion on how ISA and ESA papers are typically set, but deferred to keep the committed scope moderate and fully explainable by every team member. If added later, it would extend the `subject_tag` field in SRS Appendix B with an additional `bloom_level` enum, and add one more grouping to the existing per-topic breakdown in the Analytics Module, no new subsystem required.
- A LAN-based multi-user live quiz mode was considered at the synopsis stage and deferred for the same reason: it would require networking and concurrency that sit outside the Easy complexity budget.
- Architecture and Design references in SRS Appendix C should be updated from TBD to the values now given in Section 3.8 of this document.

5. Appendices

5.1 Glossary

Quiz, Attempt, Item, Difficulty index, RBAC: defined in SRS Appendix A and unchanged here. SAD: Software Architecture and Design specification. STRIDE: a threat modelling framework (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege).

5.2 References

PES University Software Engineering Project Guidelines, UE24CS341A. QuizSphere Deliverable 1 Synopsis and Deliverable 2 SRS v1.0.1, Team 12, Group 12.

5.3 Tools

Draw.io or PlantUML for any further diagrams, consistent with SRS Section 12. The diagrams in this document were generated with Graphviz.

Software Test Plan (STP) - QuizSphere

Online Quiz Assessment and Analytics System

Project	QuizSphere - Online Quiz Assessment and Analytics System
Version	1.0
Authors	Team 12: Arundhathi K, Aryan Revankar, Ashwin Venkatesh, Chaturya Prabhakar Reddy
Date	01-10-2026
Status	Draft (aligned with SRS v1.0 and SAD v1.0)

1. Introduction

Purpose: This document defines the test plan for QuizSphere v1.0. It outlines objectives, scope, strategy, resources, schedule and responsibilities for testing. It is written against the Deliverable 2 SRS and the Deliverable 3 Software Architecture and Design Specification (SAD), so that the three documents stay consistent.

Scope: Testing covers registration and role-based login, question bank management with CSV import/export, quiz creation and scoring configuration, the timed attempt engine (shuffling, timeout auto-submit, practice and graded modes), auto-grading with itemised result, leaderboard, item analysis, topic/student analytics including the post-attempt weak-topic summary shown to the student (FR-020, UC-06), CSV result export, the data access layer, and the role-based console menus. Subjective grading, proctoring, payments, mobile, LMS integration, networked sessions, cloud deployment and AI question authoring are excluded (SRS Section 3).

References: QuizSphere SRS v1.0 (Team 12); QuizSphere SAD v1.0 (Team 12, 01-10-2026); QuizSphere Deliverable 1 Synopsis; PES University Software Engineering Project Guidelines, UE24CS341A.

Definitions: SRS, SAD (Software Architecture and Design Specification), RTM (Requirements Traceability Matrix), FR / NFR / SEC (Functional / Non-Functional / Security Requirement), BR (Business Rule), RBAC (Role Based Access Control), DAL (Data Access Layer), STRIDE (threat modelling framework used in SAD 3.9), UT (unit test), ST (system test), CI (Continuous Integration), UAT (User Acceptance Testing). Quiz, Attempt, Item, Difficulty index: as in SRS Appendix A.

2. Test Items

Test items follow the component structure in SAD Section 3.4:

- **Auth Module** (*auth_module.**): registration, login, SHA-256 hashing, role-based session checks
- **Question Bank Module:** question CRUD, subject/difficulty/type tagging, validation, CSV import and export
- **Quiz Engine Module** (*quiz_engine.**): quiz creation and configuration, question pooling, scoring scheme setup, per-attempt shuffling, timer, practice/graded mode handling
- **Grading Module** (*grading_module.**): scoring rules including negative marking, itemised result, attempt lock
- **Analytics Module** (*analytics_module.**): leaderboard ranking, difficulty index, per-topic breakdown, post-attempt weak-topic summary, CSV result export
- **Data Access Layer:** repositories and file handlers (SQLite3 C API with flat-file fallback); the only code permitted to access storage
- **Console UI** (Student, Instructor, Admin): role-based menus, input validation at the UI boundary, confirmation prompts, countdown display
- **Data Store:** SQLite database file and CSV import/export files

3. Features to be Tested

Features mapped to SRS requirement IDs:

Req ID	Feature	Priority
FR-001	Register with unique username and role Student/Instructor	High
FR-002	Authenticate user against stored password hash	High
FR-003	Store every password as SHA-256 hash, never plaintext	High
FR-004	Instructor add/edit/delete question with subject tag, difficulty and type	High
FR-005	Reject question missing subject tag, correct answer, or with fewer than two options	High
FR-006	Bulk CSV import; skip and report rows failing validation	Medium
FR-007	Export question bank or filtered subset to CSV	Medium
FR-008	Create quiz with time limit, question count, subject/difficulty filter	High
FR-009	Scoring scheme per quiz: marks per question, optional negative marking	High
FR-010	Shuffle questions and options independently for every attempt	High
FR-011	Enforce time limit and auto-submit on timeout	High
FR-012	Navigate between questions and flag for review	Medium
FR-013	Choose practice or graded mode before attempt	High
FR-014	Practice mode: show correct answer immediately; not on leaderboard	Medium
FR-015	Graded mode: one attempt per student per quiz, logged permanently	High
FR-016	Auto-grade using quiz scoring rules including negative marking	High
FR-017	Itemised result: question, student answer, correct answer, marks	High
FR-018	Leaderboard per quiz, per topic and all time	Medium
FR-019	Per-question difficulty index	Medium
FR-020	Per-topic average score by subject tag for a student and a class; also shown to the student immediately after a graded attempt alongside the itemised result, highlighting weakest topics (UC-06)	Medium
FR-021	Export per-student result transcript to CSV	Low
NFR-001 to NFR-006	Performance (20-question quiz starts within 2 s), usability (login to quiz within 3 menu steps), reliability (timeout graded same as manual submit), maintainability (separate modules), testability (logic callable without menu), data integrity (submitted attempt not editable)	-
SEC-001 to SEC-006	No plaintext passwords; RBAC for questions/quizzes and user admin; input validation; safe CSV import; failed-login logging	-
BR-1 to BR-6	Business rules (SRS Section 9)	-
SAD design features	Error codes (INVALID_CREDENTIALS, ACCOUNT_DISABLED, ATTEMPT_ALREADY_SUBMITTED, QUIZ_NOT_FOUND, TIME_EXPIRED), transactional multi-step writes, flat-file fallback, DAL-only storage access, UI conventions (SAD 4.3-4.5)	-

4. Features Not to be Tested

- Subjective or essay question grading; webcam or lockdown-browser proctoring (out of scope)
- Payment, certification or accreditation; native mobile app; external LMS integration (out of scope)
- Real-time multi-user or networked quiz sessions; cloud deployment or multi-campus scaling (out of scope)
- AI-generated question authoring; Bloom's-taxonomy tagging and LAN multi-user mode (deferred, SAD 4.6)
- Subject-matter accuracy of question content (SRS 2.6: only format is validated)
- Internals of the SQLite3 library and any third-party SHA-256 implementation (assumed tested by their maintainers)
- API testing: not applicable, QuizSphere has no network API (SRS 4.3, 12)

5. Test Approach / Strategy

Levels

- **Unit tests (UT-):** module-level GoogleTest suites for Auth, Question Bank, Quiz Engine, Grading, Analytics and DAL, run without the console menu (NFR-005). Coverage measured with gcov/lcov.
- **Integration tests (IT-):** module interactions as drawn in the SAD component diagram, e.g. Quiz Engine to Grading to Analytics, modules to DAL to SQLite, and CSV import through DAL.
- **System tests (ST-):** end-to-end console flows for Student, Instructor and Administrator, including the sequence flows in SAD 4.2 (Attempt Quiz in graded mode, Create Quiz).
- **Acceptance tests (UAT):** Team 12 and the course instructor walk through use cases UC-01 to UC-09.

Types

- Functional (black box) and negative testing, including each named error code in SAD 4.3
- Regression testing on every push/PR through GitHub Actions
- Performance (timed run), usability (task walkthrough), reliability (timeout and retry)
- Persistence testing: transaction atomicity and SQLite versus flat-file fallback parity
- Static analysis with cppcheck

Entry Criteria: Stable build compiles in the Docker build environment (CMake), test data prepared, SQLite3/GoogleTest/cppcheck available, test environment ready.

Exit Criteria: 100% of planned test cases executed, 0 critical/high defects open, all High priority requirements verified, all acceptance criteria satisfied, cppcheck report with no unresolved errors.

5.1 Security Validation

Security tests follow the SRS security requirements and the STRIDE table in SAD Section 3.9.

STRIDE (SAD 3.9)	Test	Req / Test ID
Spoofing	Login with wrong password / unknown user returns INVALID_CREDENTIALS	FR-002, UT-AUTH-04
Tampering	Attempt to edit a submitted attempt is rejected by the Grading Module, not only hidden in the UI	NFR-006, UT-GRADE-07
Repudiation	Graded attempt is stored permanently with a timestamp	FR-015, UT-GRADE-04
Information Disclosure	No plaintext or hashed password in DB dump or logs, including after a failed login	SEC-001, SEC-006, ST-SEC-01, ST-SEC-06
Denial of Service	Malformed CSV is rejected row by row and never overwrites the question bank	SEC-005, ST-SEC-05
Elevation of Privilege	Role check at the start of every Instructor/Admin function (called directly, bypassing the menu)	SEC-002, SEC-003, UT-AUTH-07, ST-SEC-02, ST-SEC-03

- Input validation and sanitisation of every field: over-length strings (Appendix B field lengths), special characters, SQL-injection style strings, empty input (SEC-004)
- Fuzzing of console and CSV input; cppcheck for buffer overflow and unsafe C API usage

6. Test Environment

Hardware	Team members' laptops/desktops (single machine per user), Windows or Linux
Software	QuizSphere v1.0 native console application (C/C++, CMake), SQLite3 library, flat-file fallback build, Docker image pinning compiler, SQLite3, gtest and cppcheck versions
Tools	GoogleTest, gcov/lcov, cppcheck, GitHub Actions (CI), Docker, Git + GitHub, GitHub Projects and Issues (defect tracking)
Test Data	Dummy Student, Instructor and Admin accounts; seeded question bank across subjects/difficulties including unavailable (disabled) questions; valid and malformed CSV files; quizzes with and without negative marking; multiple graded and practice attempts for analytics

Database policy	Per SAD 3.7: each member tests against their own local database file; a single shared file is used only for integration testing and the final demo
Network	Not required

7. Test Schedule

Dates are planned and aligned with the sprint roadmap (WP7: Testing, CI and security validation; WP8: Final validation and report).

Milestone	Date
Test case design	20-Oct-2026
Environment setup (Docker, CI pipeline)	22-Oct-2026
Unit test execution start (alongside module delivery)	23-Oct-2026
Integration and system test execution end	04-Nov-2026
UAT	06-Nov-2026 to 09-Nov-2026
Final Test Summary Report (WP8)	12-Nov-2026

8. Test Deliverables

- Test Plan (this document)
- Test Cases (automated GoogleTest and manual)
- Test Scripts and CI workflow files
- Test Data (users, questions, CSV files)
- Test Execution Logs and GitHub Actions run logs
- Coverage report (gcov/lcov) and cppcheck report
- Defect Reports (GitHub Issues)
- Updated SRS Appendix C traceability (test case IDs)
- Test Summary Report

9. Roles and Responsibilities

Names and ownership follow the SRS work packages and the SAD component diagram.

Role	Name	Responsibility
Developer / Test Owner - Auth	Arundhathi K	Auth Module; UT-AUTH tests (incl. Admin user management), authorization and security tests
Developer / Test Owner - Question Bank	Aryan Revankar	Question Bank Module and CSV import/export; UT-QB tests
Developer / Test Owner - Quiz Engine	Ashwin Venkatesh	Quiz configuration and timed attempt engine; UT-QE and ST-QE tests
Developer / Test Owner - Grading	Ashwin Venkatesh / Chaturya Prabhakar Reddy	Grading Module (shared ownership per SAD); UT-GRADE tests
Developer / Test Owner - Analytics	Chaturya Prabhakar Reddy	Leaderboard, item analysis, analytics, CSV export; UT-ANLY and ST-ANLY tests
Data Access Layer, CI, security validation (WP7) and final validation (WP8)	All members	DAL tests, integration/system/security tests, CI maintenance, defect triage, final report

10. Risks and Mitigation

Risk	Mitigation
Delay in stable module delivery within one semester	Early smoke builds; CI build on every push; prioritise High priority requirements
SQLite file half-written if program exits mid-write (SAD 3.7)	Test that multi-step writes (attempt plus answers) are transactional; kill-process test (UT-DAL-01)
Malformed CSV corrupts question bank (SAD 3.7)	Row-level validation tests and no-overwrite tests (ST-SEC-05)
Confusing local state from shared database file (SAD 3.7)	Per-member database files; shared file only for integration and demo
Timer tests slow or flaky	Injectable time source for unit tests (NFR-005); one short real-time system test
Shuffle randomness hard to assert	Fixed seed tests plus repeated-attempt comparison
Scope creep past Easy complexity budget (SAD 3.7)	Only the SRS requirements are tested; extra features go to Open Issues

11. Assumptions and Dependencies

- Each user runs the application on their own machine with SQLite3 available at compile time
- Question content is correct; only format is validated
- GoogleTest and cppcheck are available in the CI environment
- Test data is prepared before execution
- Modules expose core logic through their declared interfaces, returning a value or a named error code (SAD 4.4)
- SAD error code ACCOUNT_DISABLED has no matching SRS requirement yet; it is tested as a design behaviour and flagged for SRS/SAD clarification

12. Suspension and Resumption Criteria

Suspend testing if:

- Build fails to compile or CI environment is unavailable for more than 4 hours
- Build is too unstable (blocks more than 30% of test cases)
- Database corruption prevents test execution

Resume testing if:

- Blocking defects are resolved and a new stable build is delivered
- Environment and test data are restored

13. Test Case Management and Traceability

The RTM maps SRS requirements to the SAD module and to test cases. Test IDs follow the SRS Appendix C convention: UT-AUTH, UT-QB, UT-QE, UT-GRADE, UT-ANLY, UT-DAL for module unit tests; ST- for system tests; plus PERF, USE, REL, MNT, IT, SEC for other types. Once executed, the IDs below will replace the sample entries in SRS Appendix C.

Req ID	SAD Module	Test Case IDs	Type
FR-001	Auth	UT-AUTH-02, UT-AUTH-03, UT-AUTH-09	Unit / Negative
FR-002	Auth, DAL	UT-AUTH-01, UT-AUTH-04	Unit
FR-003	Auth	UT-AUTH-05	Unit / Code review
FR-004	Question Bank	UT-QB-01, UT-QB-02, UT-QB-03	Unit
FR-005	Question Bank	UT-QB-04, UT-QB-05, UT-QB-06	Negative
FR-006	Question Bank	UT-QB-07, UT-QB-08	Functional
FR-007	Question Bank	UT-QB-09	Functional
FR-008	Quiz Engine	UT-QE-02, UT-QE-03	Unit

Req ID	SAD Module	Test Case IDs	Type
FR-009	Quiz Engine	UT-QE-04, UT-QE-05	Unit
FR-010	Quiz Engine	UT-QE-01, ST-QE-01	Unit / System
FR-011	Quiz Engine	UT-QE-06, ST-QE-02	Unit / System
FR-012	Quiz Engine, UI	ST-QE-03	System
FR-013	Quiz Engine	ST-QE-04	System
FR-014	Quiz Engine	ST-QE-05	System
FR-015	Grading, Quiz Engine	UT-GRADE-04, ST-QE-06	Negative
FR-016	Grading	UT-GRADE-01, UT-GRADE-02, UT-GRADE-03	Unit / System
FR-017	Grading	UT-GRADE-06, ST-GRADE-01	Unit / System
FR-018	Analytics	UT-ANLY-03, UT-ANLY-04	Unit
FR-019	Analytics, DAL	UT-ANLY-01	Unit
FR-020	Analytics	UT-ANLY-02, ST-ANLY-01	Unit / System
FR-021	Analytics, DAL	UT-ANLY-05, ST-ANLY-02	Unit / Functional
NFR-001	Quiz Engine	PERF-01	Timed run
NFR-002	Console UI	USE-01	Walkthrough
NFR-003	Quiz Engine, Grading	REL-01	Negative / Integration
NFR-004	All modules	MNT-01	Code review
NFR-005	All modules	Entire UT suite runs without the console menu	Unit
NFR-006	Grading, DAL	UT-GRADE-07, IT-01	Unit / Integration
UC-09 / SEC-003	Auth, Console UI	UT-AUTH-08, ST-ADM-01	Unit / System
UC-07, UC-08 (Admin)	Analytics, Console UI	ST-ANLY-03, ST-ANLY-04	System
Admin question bank oversight and system reports (SRS 2.3)	Question Bank, Console UI	ST-ADM-02	System
SEC-001 to SEC-006	Auth, DAL, Question Bank, UI	ST-SEC-01 to ST-SEC-06, UT-AUTH-07	Authorization / Negative
BR-1, BR-2	Quiz Engine	UT-QE-07, UT-QE-08	Negative
BR-3	Grading	UT-GRADE-04	Negative
BR-4	Analytics	UT-ANLY-06	Unit
BR-5	Grading	UT-GRADE-02	Unit
BR-6	Question Bank	UT-QB-10, ST-SEC-02	Functional
SAD 4.3 error codes	Auth, Grading	UT-AUTH-04, UT-AUTH-06, UT-GRADE-04, UT-GRADE-08, UT-GRADE-09	Unit
SAD 3.7 / 3.5	DAL	UT-DAL-01 (atomic write), UT-DAL-02 (flat-file parity), UT-DAL-03 (DAL-only storage access)	Unit / Review
SAD 4.4 logging	Auth, Question Bank	ST-SEC-06 (failed login), UT-QB-11 (rejected CSV rows logged with timestamp and reason)	Unit / System
SAD 4.4 / 4.5	Console UI	ST-UI-01 to ST-UI-04	System

13.1 Key Test Case Specifications (sample)

ID	Scenario	Steps / Input	Expected Result
UT-AUTH-01	Valid login	Login with correct username and password	Session{user_id, role, logged_in_at} returned
UT-AUTH-04	Wrong password	Login with incorrect password	INVALID_CREDENTIALS; failure logged with timestamp, no password
UT-AUTH-05	Hash storage	Inspect DB after registration	Only SHA-256 hash present; no plaintext
UT-AUTH-06	Disabled account	Login with a disabled account (SAD design behaviour)	ACCOUNT_DISABLED returned
UT-AUTH-07	Direct privilege call	Call an Instructor-only function with a Student session	Denied at function level, not only in the UI
UT-AUTH-08	Admin manages users	Admin creates a user, changes a role, removes a user	Account and role changes saved; removal needs confirmation
UT-AUTH-09	Admin self-registration	Register new user with role Admin	Rejected; only Student or Instructor allowed (FR-001)
ST-ADM-01	Admin user flow	Login as Admin, run UC-09 through the console menu	Users and roles updated; Student/Instructor cannot reach this menu
ST-ADM-02	Admin oversight	Admin opens question bank import/export and system reports	Actions available and results displayed (SRS 2.3)
ST-ANLY-04	Admin export	Admin exports a per-student result transcript (UC-08)	CSV transcript file written
ST-ANLY-03	Admin analytics	Admin views item analysis and topic report (UC-07)	Reports displayed when at least one graded attempt exists
UT-QB-05	Missing subject tag	Add question without subject tag	Question rejected
UT-QB-10	Disable question	Instructor disables a question, then creates a new quiz	Question no longer offered for new quizzes (BR-6)
UT-QB-11	Rejected row logging	Import CSV containing invalid rows	Each rejected row logged with timestamp and reason (SAD 4.4)
UT-QB-08	CSV with bad rows	Import CSV with 2 invalid rows of 10	8 imported; 2 skipped and reported
UT-QE-01	Shuffle independence	Two attempts of same quiz	Question and option order differ per attempt
UT-QE-08	Zero time limit	Create quiz with 0 minute limit	Quiz creation rejected (BR-2)
ST-QE-02	Timeout auto-submit	Let timer expire mid-attempt	Auto-submitted and graded; countdown was visible
UT-GRADE-01	Basic grading	Submit answers with +4 correct, -1 wrong	Score matches scoring scheme; itemised result produced
UT-GRADE-02	Negative floor	Wrong answer where penalty exceeds marks	Question score never below zero (BR-5)
UT-GRADE-04	Second graded attempt	Re-submit graded attempt for same quiz/student	ATTEMPT_ALREADY_SUBMITTED; first attempt unchanged
UT-GRADE-08	Unknown quiz	Grade attempt for a non-existent quiz_id	QUIZ_NOT_FOUND
UT-GRADE-09	Late submission	Grade attempt after expiry	Graded and flagged auto-submitted (TIME_EXPIRED)
UT-ANLY-01	Difficulty index	4 attempts, 3 correct on an item	Index = 75%
UT-ANLY-02	Topic breakdown	Student with low Algebra and high Geometry scores	Average per subject tag correct; weakest topic identified
ST-ANLY-01	Post-attempt summary	Finish a graded attempt	Itemised result and weak-topic summary shown together (UC-06)
UT-ANLY-06	Practice excluded	Complete a practice attempt	Not on leaderboard; not counted in analytics (BR-4)

ID	Scenario	Steps / Input	Expected Result
UT-DAL-01	Atomic write	Terminate program mid-save of attempt plus answers	Either both stored or neither; no half-written attempt
ST-UI-02	Destructive confirmation	Delete a question or user	Yes/no prompt shown before deletion
ST-SEC-05	Malformed CSV	Import corrupted file	Question bank unchanged
PERF-01	Load time	Load and start 20-question quiz	Starts within 2 seconds

14. Test Metrics and Reporting

Metrics collected:

- % test cases executed; % passed / failed
- Defect density and defect aging (GitHub Issues)
- Requirement coverage (RTM) and code coverage (gcov/lcov)
- cppcheck findings and CI pipeline pass rate

Reports: daily execution status during the execution window, GitHub Actions workflow logs, and the final Test Summary Report.

15. Approvals

Role	Name	Signature / Date
Developer / Test Owner - Auth	Arundhathi K	
Developer / Test Owner - Question Bank	Aryan Revankar	
Developer / Test Owner - Quiz Engine and Grading	Ashwin Venkatesh	
Developer / Test Owner - Grading and Analytics	Chaturya Prabhakar Reddy	
Course Instructor	Dr. Sapna V M	